BÁO CÁO BÀI THỰC HÀNH SỐ 4
MÔN PHÁT TRIỂN ỨNG DỤNG IOT

Giảng viên	: Nguyễn Quốc Uy
Họ và tên sinh viên	: Trương Đức Mạnh
Mã sinh viên	: B22DCPT162
Nhóm lớp	: 02

Hà Nội - 2026

MỤC LỤC

Chương I: Giới thiệu đề tài 4
1.1 Đặt vấn đề 4
1.2 Mục tiêu đề tài 4
1.3 Phạm vi hệ thống 4
1.4 Công nghệ chủ chốt 4
Chương II: Thiết kế giao diện 5
2.1. Dashboard 5
2.2. Data Sensor 5
2.3. Action History 6
2.4. Profile 7
Chương III: Thiết kế hệ thống 8
3.1 Sơ đồ tuần tự 8
3.2 Thiết kế cơ sở dữ liệu 11
Chương IV: Các thiết bị sử dụng 13
4.1 Phần cứng IoT 13
4.2 Hạ tầng truyền thông 14
Chương V: Triển khai hệ thống 14
5.1. Triển khai Backend 14
5.2 Triển khai Frontend 18
Chương VI: Postman 19
6.1 Action Hitory 19
6.2 Data Sensor 19
6.3 Device Control 20

MỤC LỤC HÌNH ẢNH

Hình 2.1 Giao diện Dashboard.....5
Hình 2.2 Giao diện Data Sensor 6
Hình 2.3 Giao diện Action History 7
Hình 2.4 Giao diện Profile 8
Hình 3.1 Luồng theo dõi cảm biến.....8
Hình 3.2 Luồng điều khiển thiết bị 9
Hình 3.3 Luồng tra cứu lịch sử thao tác.....10
Hình 3.4 Luồng tra cứu dữ liệu cảm biến 11
Hình 3.5 Thiết kế cơ sở dữ liệu.....12
Hình 5.1 API điều khiển thiết bị 15
Hình 5.2 API tra cứu lịch sử thao tác.....16
Hình 5.3 API tra cứu dữ liệu cảm biến 18
Hình 6.1 Request Params API api/action-history/search.....19
Hình 6.2 Response API api/action-history/search 19
Hình 6.3 Request Params API api/data-sensor/search.....19
Hình 6.4 Response API api/data-sensor/search 20
Hình 6.5 Request Params API api/device/control.....20
Hình 6.6 Response API api/device/control 20

Chương I: Giới thiệu đề tài

1.1 Đặt vấn đề

Trong kỷ nguyên Internet vạn vật, việc tự động hóa và giám sát môi trường sống trở nên thiết yếu nhằm nâng cao tiện nghi và tiết kiệm năng lượng. Đề tài tập trung giải quyết vấn đề quản lý từ xa các thông số môi trường và thiết bị điện trong một phòng thông minh.

1.2 Mục tiêu đề tài

- Giám sát thời gian thực: Thu thập và hiển thị dữ liệu nhiệt độ, độ ẩm và ánh sáng từ các cảm biến.
- Điều khiển linh hoạt: Bật/tắt các thiết bị như quạt, điều hòa và đèn thông qua giao diện web.
- Quản lý dữ liệu: Lưu trữ và tra cứu lịch sử đo đạc của cảm biến và nhật ký thao tác điều khiển thiết bị.

1.3 Phạm vi hệ thống

Hệ thống bao gồm một Dashboard điều khiển tập trung và các trang quản lý chuyên biệt:

- Dashboard: Hiển thị biểu đồ thông số theo thời gian và các nút điều khiển thiết bị (Fan, Air Condition, Light Bulb).
- Data Sensor: Lưu trữ lịch sử giá trị từ các cảm biến như LM35, DHT22, Quang trở.
- Action History: Nhật ký ghi lại trạng thái thực hiện lệnh (SUCCESS/FAILED) và hành động của người dùng.

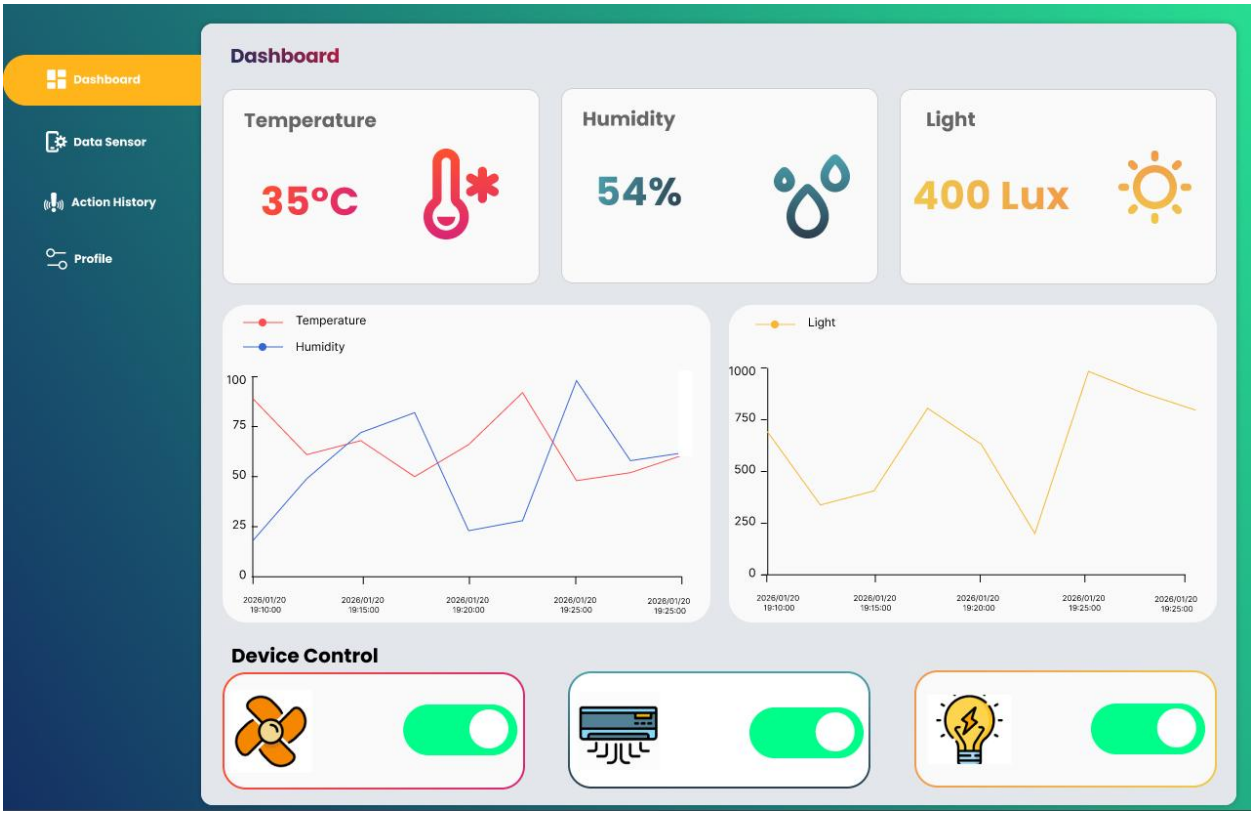
1.4 Công nghệ chủ chốt

- Backend: Spring Boot, Spring Data JPA, Spring WebSocket, Spring Integration MQTT.
- Frontend: React, TypeScript, Vite, Recharts, STOMP/SockJS.
- Database: MySQL.
- Giao thức truyền thông IoT: MQTT.

Chương II: Thiết kế giao diện

2.1. Dashboard

Trang Dashboard hiển thị trực quan các giá trị được thu thập từ cảm biến. Các biểu đồ thể hiện giá trị dữ liệu thay đổi theo thời gian và các nút bấm để người dùng dễ dàng tương tác với các thiết bị hơn



Hình 2.1 Giao diện Dashboard

2.2. Data Sensor

- Trang Data Sensor lưu trữ lại các bản ghi dữ liệu của cảm biến theo thời gian thực

- Bảng dữ liệu gồm 4 cột:

+ ID: Số thứ tự của bản ghi

+ Sensor Name: Tên của cảm biến

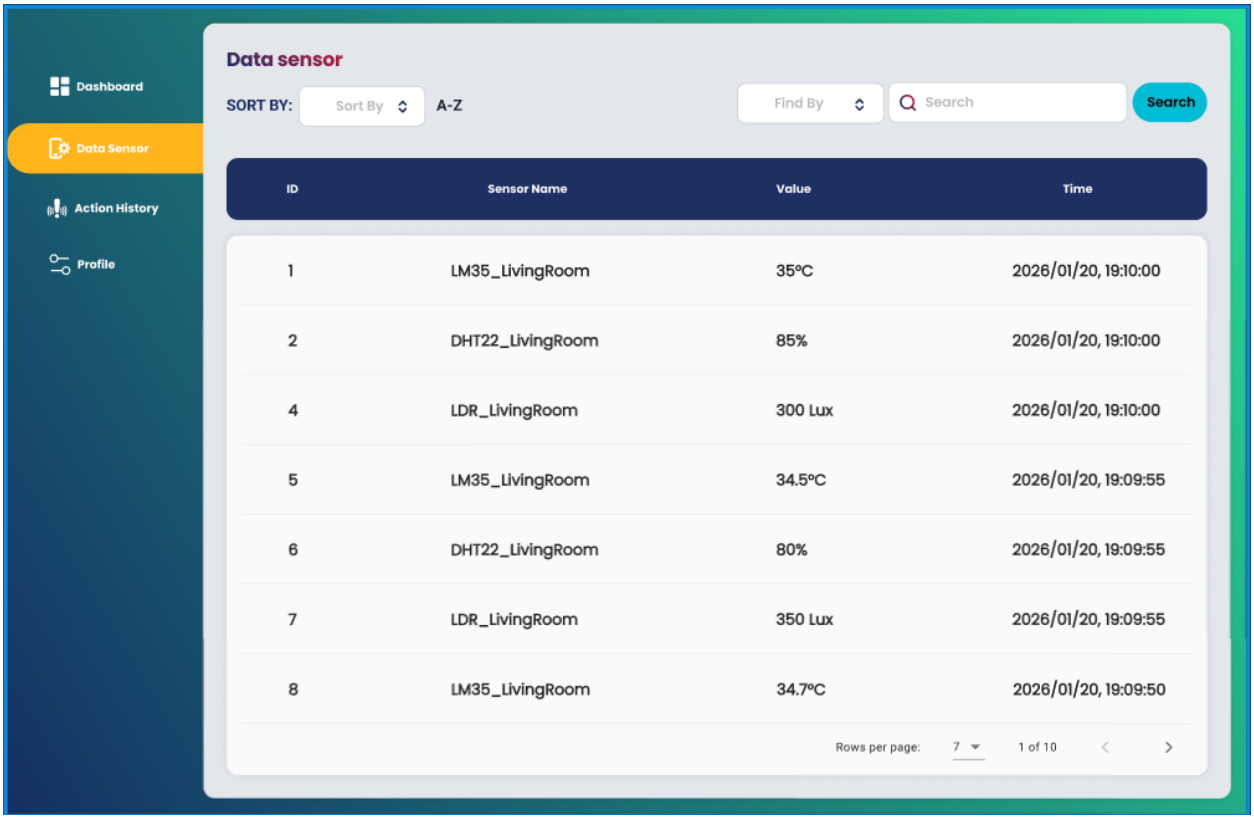
+ Value: Giá trị của cảm biến được ghi nhận

+ Time: Thời gian ghi nhận giá trị của cảm biến

- Trang Data Sensor có một khung để tìm kiếm theo giá trị người dùng muốn

- Nút Sort cho phép sắp xếp dữ liệu theo tăng dần hoặc giảm dần.

- Trang Data Sensor có phân trang, có tối đa 20 bản ghi



Hình 2.2 Giao diện Data Sensor

2.3. Action History

- Trang Action History lưu lại các bản ghi lịch sử bật tắt của các thiết bị.

- Bảng gồm 5 cột:

+ ID: Số thứ tự của bản ghi

+ Device Name: Tên của thiết bị

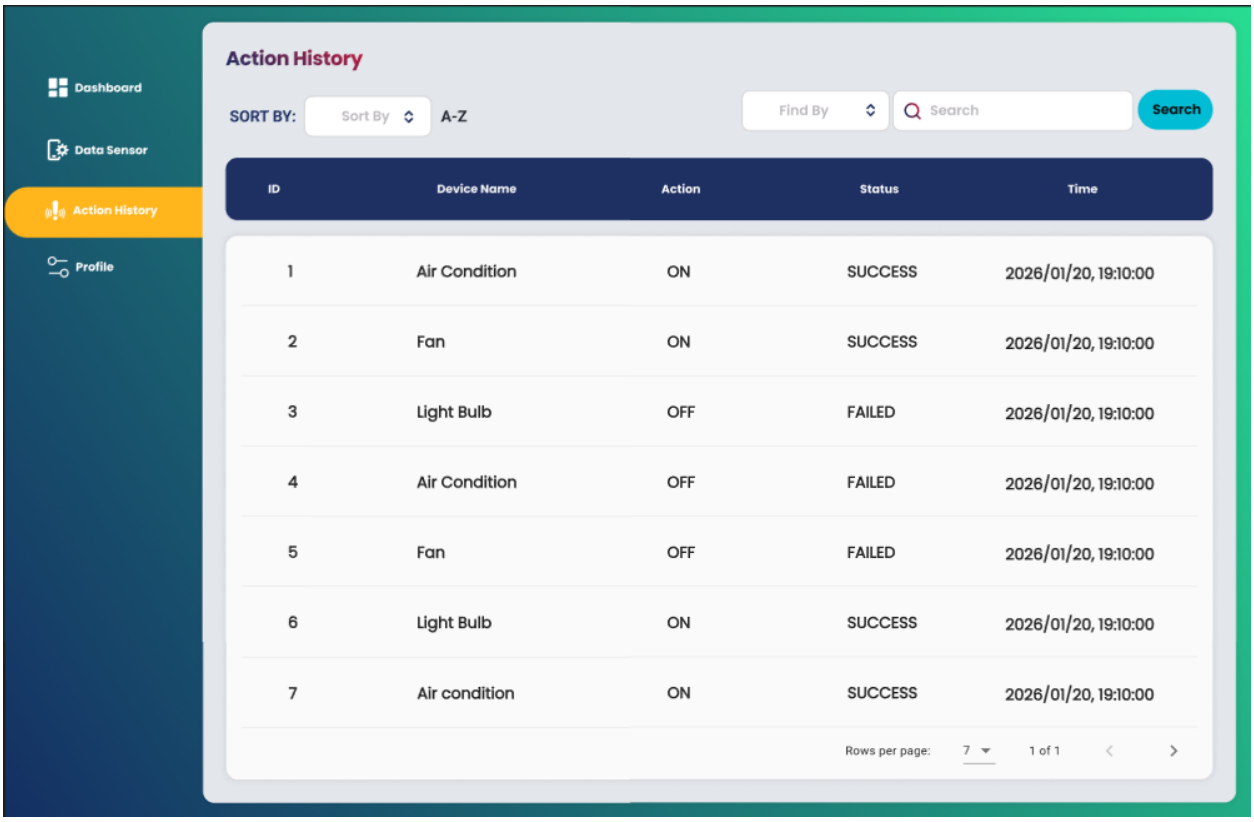
+ Action: Hành động của người dùng

+ Status: Trạng thái của thiết bị sau hành động của người dùng

+ Time: Thời gian ghi nhận của bản ghi

- Nút Sort cho phép sắp xếp dữ liệu theo tăng dần hoặc giảm dần.

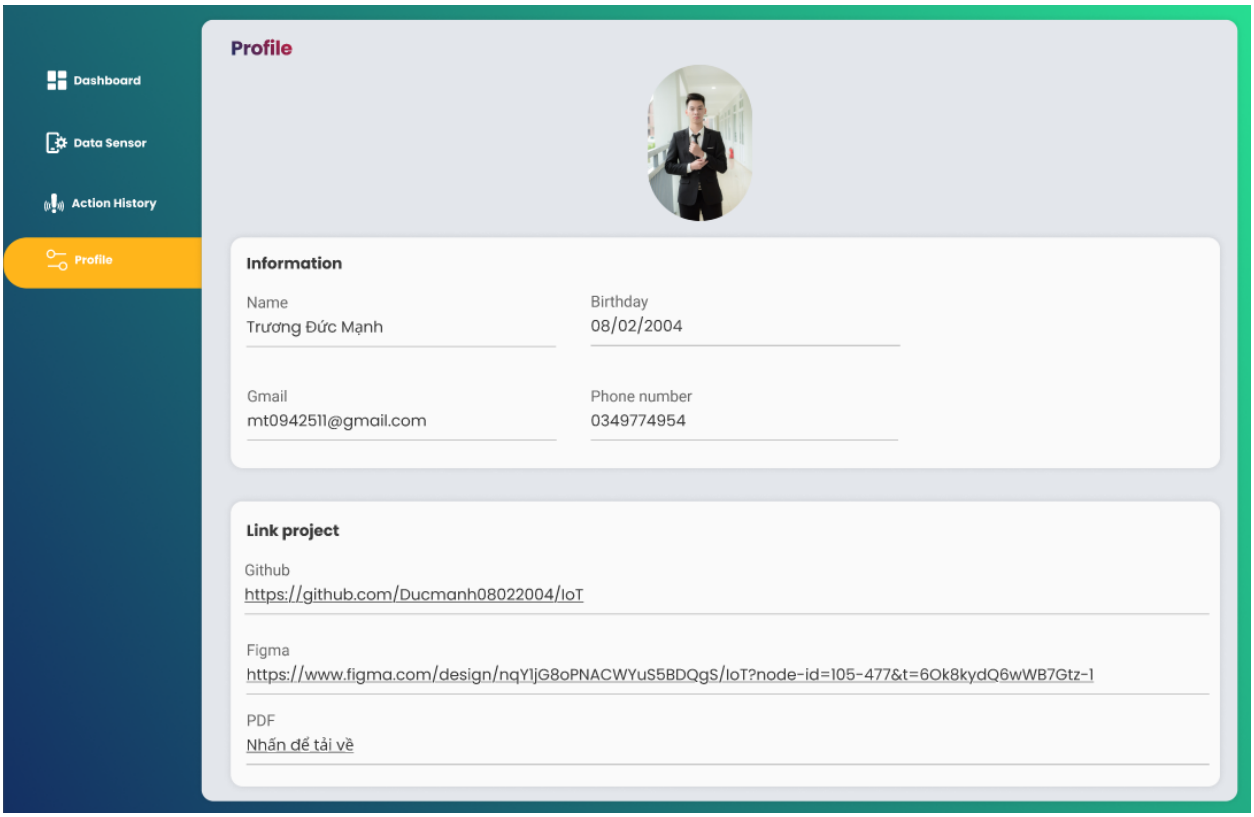
- Trang Data Sensor có phân trang, có tối đa 20 bản ghi



Hình 2.3 Giao diện Action History

2.4. Profile

- Trang Profile hiển thị thông tin cá nhân và các đường link đến Figma, Github và tài liệu PDF mô tả của dự án.

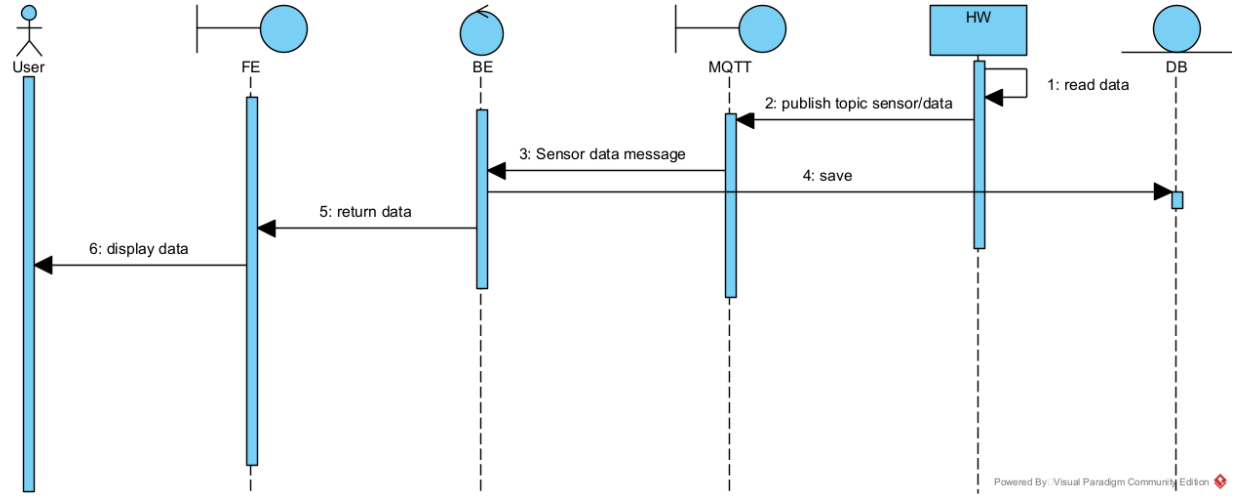


Hình 2.4 Giao diện Profile

Chương III: Thiết kế hệ thống

3.1 Sơ đồ tuần tự

- Luồng theo dõi cảm biến thời gian thực

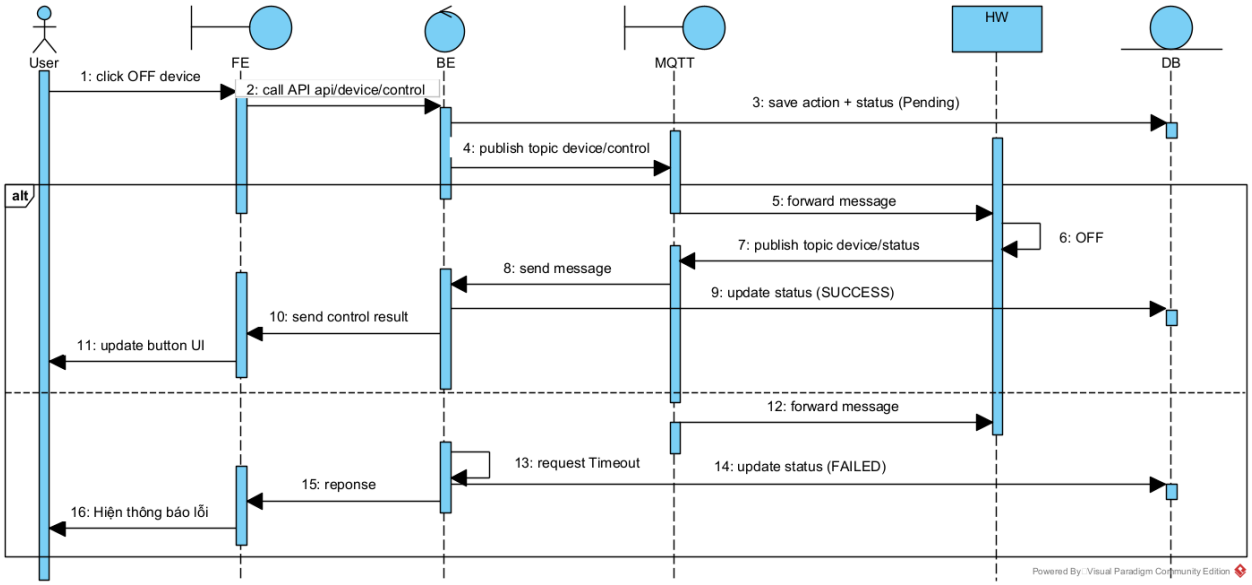


Hình 3.1 Luồng theo dõi cảm biến

+ Biểu đồ này mô tả luồng dữ liệu tự động từ phần cứng đẩy lên hệ thống theo thời gian thực.

- Bước 1: Sensor đọc từ liệu từ chính nó.
- Bước 2: Sensor thực hiện publish dữ liệu vào topic sensor/data thông qua MQTT Broker.
- Bước 3: BE (đã subscribe topic) nhận được tin nhắn chứa dữ liệu cảm biến.
- Bước 4: BE thực hiện lưu trữ (Save) các dữ liệu này vào Database (DB).
- Bước 5: Đồng thời, BE đẩy dữ liệu về cho FE.
- Bước 6: FE cập nhật và hiển thị dữ liệu mới nhất lên màn hình để người dùng theo dõi trực tiếp.

- Luồng điều khiển thiết bị



Hình 3.2 Luồng điều khiển thiết bị

+ Luồng chính (Thành công):

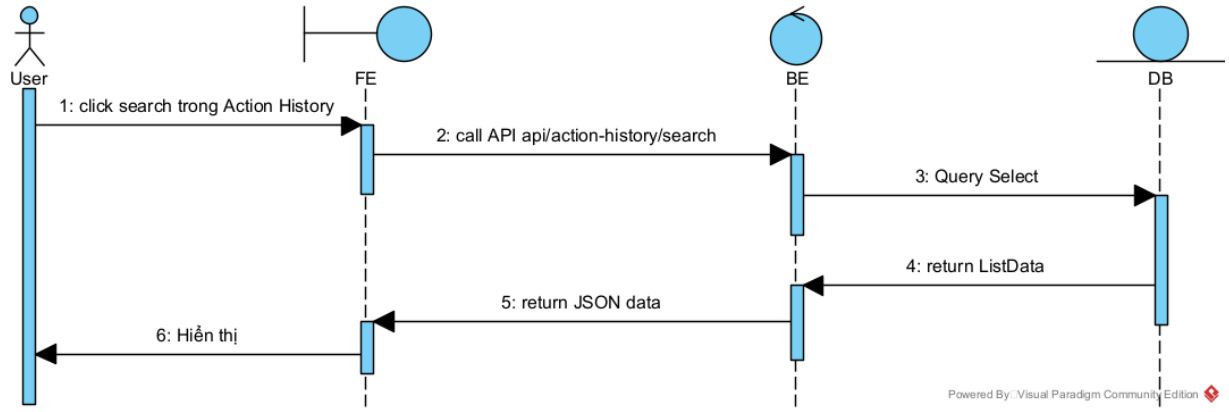
- Bước 1 - 2: User nhấn nút "OFF" thiết bị, FE gọi API điều khiển đến BE.
- Bước 3: BE lưu trạng thái yêu cầu vào DB với trạng thái ban đầu là Pending.

- Bước 4 - 5: BE publish (gửi) một tin nhắn vào topic device/control thông qua MQTT Broker. Broker sau đó forward (chuyển tiếp) thông điệp này đến thiết bị phần cứng (HW).
- Bước 6: Thiết bị thực hiện lệnh.
- Bước 7 - 8: HW phản hồi lại trạng thái mới thông qua MQTT, gửi tin nhắn ngược lại cho BE qua topic device/status.
- Bước 9: BE cập nhật trạng thái mới của thiết bị vào DB.
- Bước 10 - 11: BE gửi kết quả điều khiển về cho FE để cập nhật lại trạng thái nút bấm trên giao diện người dùng.

+ Luồng thay thế (Xử lý lỗi):

- Bước 12: MQTT chuyển tiếp thông điệp đến thiết bị.
- Bước 13-14: Nếu có vấn đề, hệ thống ghi nhận trạng thái thất bại vào DB.
- Bước 15-16: BE gửi phản hồi cho FE, và FE cập nhật trạng thái nút hiển thị thông báo cho người dùng.

- Luồng tra cứu lịch sử thao tác



Hình 3.3 Luồng tra cứu lịch sử thao tác

+ Bước 1: User nhấn nút "Search" trong giao diện Action History trên Frontend (FE).

+ Bước 2: FE gửi yêu cầu gọi API api/action-history/search đến Backend (BE).

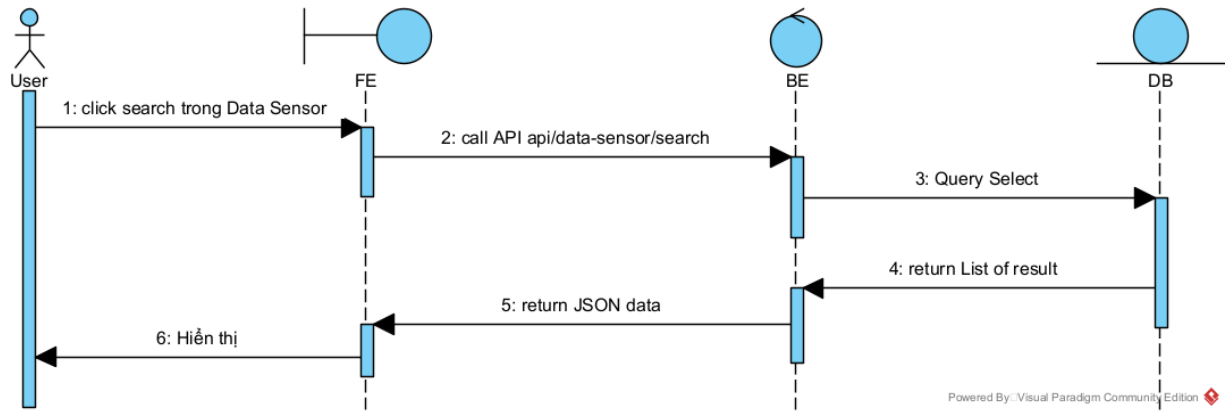
+ Bước 3: BE thực hiện lệnh Query Select xuống Database (DB) để lấy dữ liệu lịch sử.

+ Bước 4: DB trả kết quả danh sách dữ liệu (return ListData) cho BE.

+ Bước 5: BE xử lý và trả về dữ liệu dưới dạng JSON cho FE.

+ Bước 6: FE nhận dữ liệu và hiển thị (Render) lên màn hình cho User.

- Luồng tra cứu dữ liệu cảm biến



Hình 3.4 Luồng tra cứu dữ liệu cảm biến

+ Bước 1: User nhấn nút "Search" trong màn hình Data Sensor trên FE.

+ Bước 2: FE gọi API api/data-sensor/search gửi đến BE.

+ Bước 3: BE truy vấn cơ sở dữ liệu (DB) bằng lệnh Query Select.

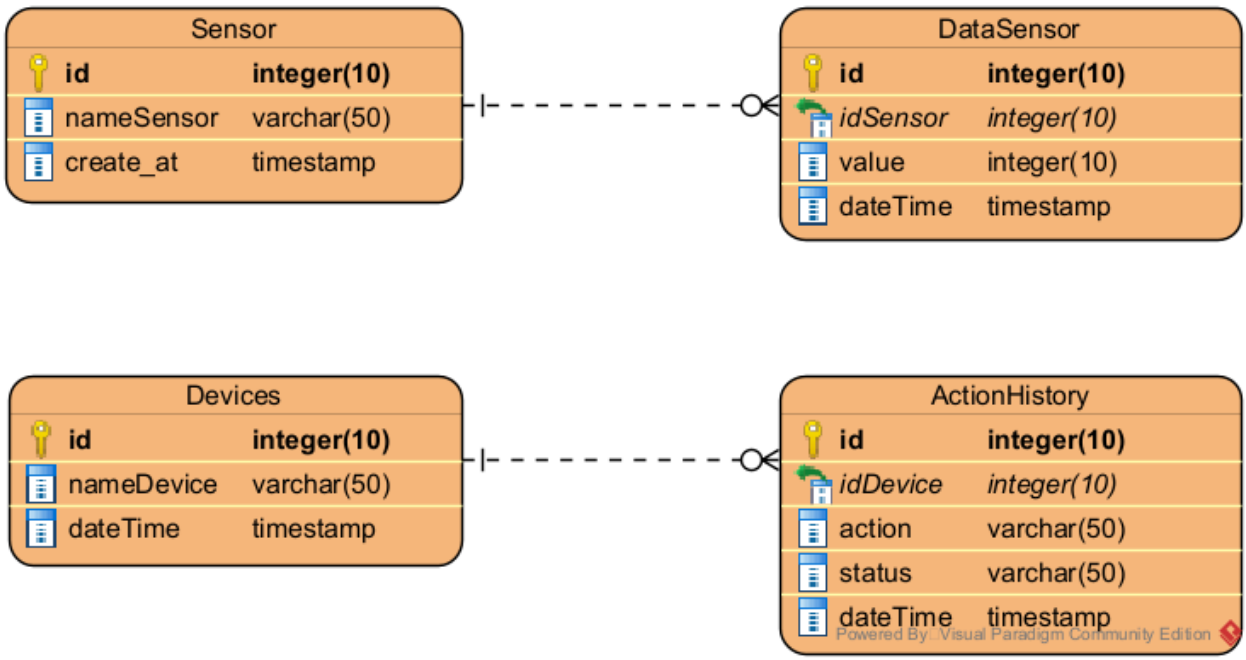
+ Bước 4: DB trả về danh sách kết quả đo đạc của cảm biến cho BE.

+ Bước 5: BE đóng gói và trả dữ liệu JSON về cho FE.

+ Bước 6: FE thực hiện hiển thị các thông số lên giao diện cho User.

3.2 Thiết kế cơ sở dữ liệu

- Cơ sở dữ liệu gồm 4 bảng chính:



Hình 3.5 Thiết kế cơ sở dữ liệu

+ Bảng Sensor: Lưu trữ thông tin danh mục các loại cảm biến.

- id (PK): Mã định danh duy nhất của cảm biến.
- nameSensor: Tên của cảm biến.
- create_at: Thời gian khởi tạo cảm biến trong hệ thống.

+ Bảng DataSensor: Lưu trữ các giá trị đo đạc thực tế từ cảm biến.

- id (PK): Mã định danh bản ghi dữ liệu.
- idSensor (FK): Khóa ngoại liên kết tới bảng Sensor.
- value: Giá trị đo được từ cảm biến.
- dateTime: Thời điểm ghi nhận dữ liệu.

+ Bảng Devices: Lưu trữ thông tin về các thiết bị có thể điều khiển.

- id (PK): Mã định danh duy nhất của thiết bị.
- nameDevice: Tên thiết bị.
- dateTime: Thời gian thiết bị được thêm vào hệ thống.

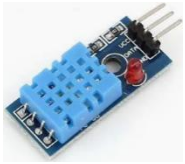
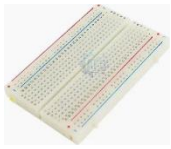
+ Bảng ActionHistory: Lưu trữ nhật ký các thao tác điều khiển thiết bị.

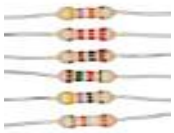
- id (PK): Mã định danh bản ghi lịch sử.
- idDevice (FK): Khóa ngoại liên kết tới bảng Devices.
- action: Hành động thực hiện.

- status: Trạng thái của hành động.
- dateTime: Thời điểm thực hiện thao tác.

Chương IV: Các thiết bị sử dụng

4.1 Phần cứng IoT

Hình ảnh	Tên thiết bị
	Vi điều khiển ESP32
	Quang trở
	Cảm biến độ ẩm/ nhiệt độ DHT22
	Led
	Breadboard
	Dây nối

	Điện trở

4.2 Hạ tầng truyền thông

- MQTT Broker nội bộ chạy tại cổng 1883/ 1884/ 1885.

- Kênh WebSocket để đồng bộ dữ liệu với frontend.

Chương V: Triển khai hệ thống

5.1. Triển khai Backend

5.1.1. Cấu trúc lớp

- Backend được chia thành các lớp:

    + controller: xử lý request/response API.

    + service: xử lý nghiệp vụ.

    + repository: truy vấn dữ liệu.

    + specification: truy vấn động theo điều kiện lọc.

    + mqtt/config: tích hợp MQTT, WebSocket, CORS.

5.1.2. Các API đã triển khai

- POST /api/device/control

    + Nhận tham số led và action.

    + Lưu lịch sử Pending.

    + Gửi lệnh MQTT.

    + Update Status vào Database

```java
public class DeviceController {

    private static final long DEVICE_ACK_TIMEOUT_SECONDS = 5;

    private final MqttConfig.MqttGateway mqttGateway;
    private final actionHistoryRepository actionHistoryRepo;
    private final deviceRepository deviceRepo;
    private final ActionHistoryTimeoutService actionHistoryTimeoutService;

    @PostMapping("/control")
    public String control(@RequestParam String led, @RequestParam String action) {
        Device device = deviceRepo.findByNameDevice(led)
                .orElseThrow(() -> new RuntimeException("Không tìm thấy thiết bị: " + led));

        ActionHistory history = new ActionHistory();
        history.setDevice(device);
        history.setAction(action);
        history.setStatus(status: "Pending");
        history.setDateTime(LocalDateTime.now());
        actionHistoryRepo.save(history);

        actionHistoryTimeoutService.schedulePendingToFailed(history.getId(), DEVICE_ACK_TIMEOUT_SECONDS);

        String payload = led + " " + action;
        mqttGateway.sendToMqtt(payload, topic: "device/control");

        return "Sent " + payload + " (Status: Pending)";
    }
}
```

Hình 5.1 API điều khiển thiết bị

- POST /api/action-history/search

+ Hỗ trợ filter + sort + pagination.

```java
public class ActionHistoryService {

    private final actionHistoryRepository repository;
        private static final int MAX_PAGE_SIZE = 100;
        private static final Set<String> ALLOWED_SORT_FIELDS = Set.of(
                        "id",
                        "device.nameDevice",
                        "action",
                        "status",
                        "dateTime"
        );

    public Page<actionHistoryResponse> search(
            actionHistoryFilterRequest request,
            int page,
            int size,
            String sortBy,
            String direction
    ) {

        actionHistoryFilterRequest safeRequest = request == null
                ? new actionHistoryFilterRequest()
                : request;

        int safePage = Math.max(page, 0);
        int safeSize = size <= 0 ? 10 : Math.min(size, MAX_PAGE_SIZE);
        String safeSortBy = ALLOWED_SORT_FIELDS.contains(sortBy) ? sortBy : "dateTime";

        Sort sort = "desc".equalsIgnoreCase(direction)
                ? Sort.by(safeSortBy).descending()
                : Sort.by(safeSortBy).ascending();

        Pageable pageable = PageRequest.of(safePage, safeSize, sort);

        Page<ActionHistory> result = repository.findAll(
                ActionHistorySpecification.filter(
                        safeRequest.getNameDevice(),
                        safeRequest.getAction(),
                        safeRequest.getStatus(),
                        safeRequest.getDate()
                ),
                pageable
        );
        return result.map(action -> new actionHistoryResponse(
                action.getId(),
                action.getDevice().getNameDevice(),
                action.getAction(),
                action.getStatus(),
                action.getDateTime()
        ));
    }
}
```

Hình 5.2 API tra cứu lịch sử thao tác

- POST /api/data-sensor/search

+ Hỗ trợ filter + sort + pagination.

```java
public class DataSensorService {

    private final dataSensorRepository repository;
        private static final int MAX_PAGE_SIZE = 100;
        private static final Set<String> ALLOWED_SORT_FIELDS = Set.of(
                        "id",
                        "sensor.nameSensor",
                        "value",
                        "dateTime"
        );

    public Page<dataSensorResponse> search(
            dataSensorFilterRequest request,
            int page,
            int size,
            String sortBy,
            String direction
    ) {

        dataSensorFilterRequest safeRequest = request == null
                ? new dataSensorFilterRequest()
                : request;

        int safePage = Math.max(page, 0);
        int safeSize = size <= 0 ? 10 : Math.min(size, MAX_PAGE_SIZE);
        String safeSortBy = ALLOWED_SORT_FIELDS.contains(sortBy) ? sortBy : "dateTime";

        Sort sort = "desc".equalsIgnoreCase(direction)
                ? Sort.by(safeSortBy).descending()
                : Sort.by(safeSortBy).ascending();

        Pageable pageable = PageRequest.of(safePage, safeSize, sort);

        Page<DataSensor> result = repository.findAll(
                DataSensorSpecification.filter(
                        safeRequest.getNameSensor(),
                        safeRequest.getValue(),
                        safeRequest.getDateTime()
                ),
                pageable
        );
        return result.map(action -> new dataSensorResponse(
                action.getId(),
                action.getSensor().getNameSensor(),
                action.getValue(),
                action.getDateTime()
        ));
    }
}
```

Hình 5.3 API tra cứu dữ liệu cảm biến

5.2 Triển khai Frontend

5.2.1 Kiến trúc giao diện

- Router điều hướng 4 trang: Dashboard, Data Sensor, Action History, Profile.
- Lazy loading cho các trang để tối ưu tải ban đầu.
- Component tái sử dụng: DataTable, CustomSelect, ChartCard, DeviceControlCard.

5.2.2 Tầng tích hợp dữ liệu

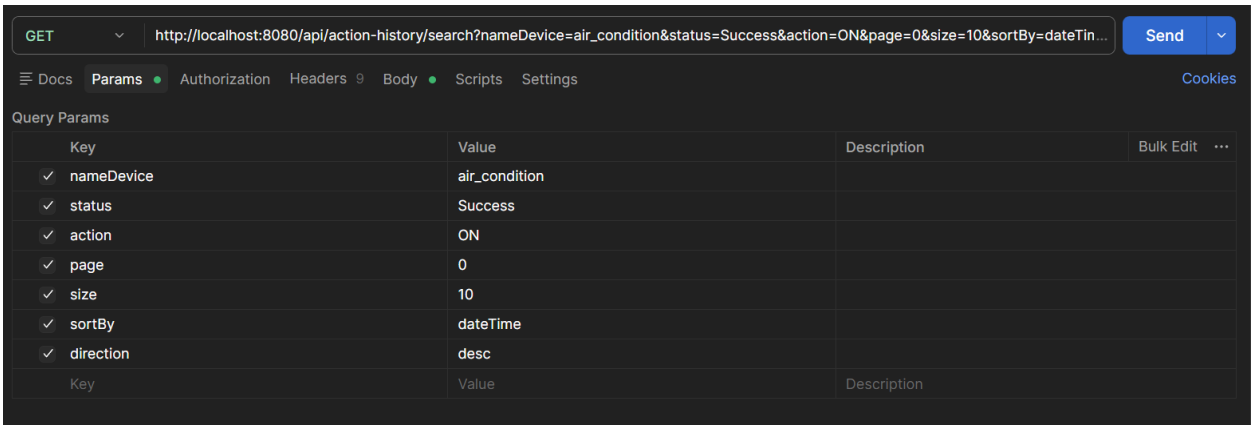
- REST API:
    + Tìm kiếm dữ liệu cảm biến.
    + Tìm kiếm lịch sử thao tác.
    + Gửi lệnh điều khiển thiết bị.
- WebSocket:
    + Nhận dữ liệu cảm biến realtime.
    + Nhận trạng thái thiết bị realtime.

5.2.3 Xử lý trải nghiệm người dùng

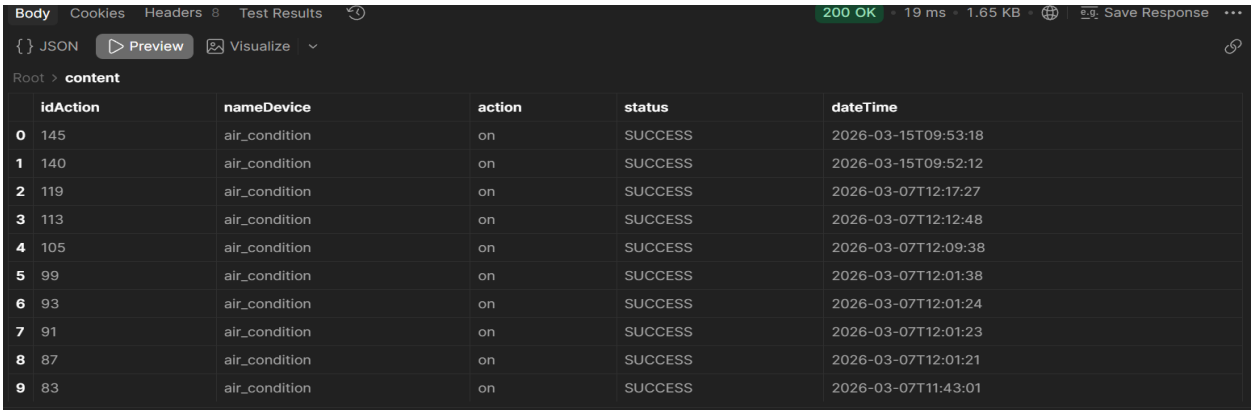
- Dashboard hiển thị realtime và fallback polling khi mất kết nối.
- Trạng thái điều khiển có Pending để phản ánh độ trễ thực tế của thiết bị.
- Bảng dữ liệu có phân trang, tìm kiếm, sắp xếp và trạng thái loading/error rõ ràng.

Chương VI: Postman

6.1 Action Hitory



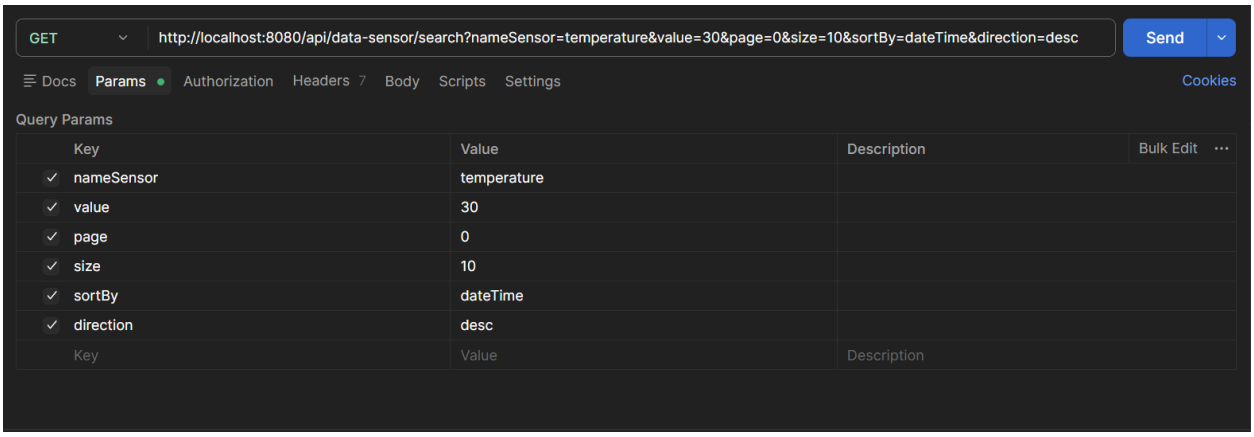
Hình 6.1 Request Params API api/action-history/search



	idAction	nameDevice	action	status	dateTime
0	145	air_condition	on	SUCCESS	2026-03-15T09:53:18
1	140	air_condition	on	SUCCESS	2026-03-15T09:52:12
2	119	air_condition	on	SUCCESS	2026-03-07T12:17:27
3	113	air_condition	on	SUCCESS	2026-03-07T12:12:48
4	105	air_condition	on	SUCCESS	2026-03-07T12:09:38
5	99	air_condition	on	SUCCESS	2026-03-07T12:01:38
6	93	air_condition	on	SUCCESS	2026-03-07T12:01:24
7	91	air_condition	on	SUCCESS	2026-03-07T12:01:23
8	87	air_condition	on	SUCCESS	2026-03-07T12:01:21
9	83	air_condition	on	SUCCESS	2026-03-07T11:43:01

Hình 6.2 Response API api/action-history/search

6.2 Data Sensor



Hình 6.3 Request Params API api/data-sensor/search

Body Cookies Headers 8 Test Results 200 OK 15 ms 656 B Save Response

{ } JSON Preview Visualize

Root > content

	idData	nameSensor	value	dateTime
0	9	Temperature	30	2024-03-20T12:00:00

Hình 6.4 Response API api/data-sensor/search

6.3 Device Control

POST http://localhost:8080/api/device/control?led=light_bulb&action=off Try

Params Headers Body

Query Params

Key	Value	Description	Bulk Edit
✓ led	light_bulb	Target device: light_bulb | light_bulbn | air_condition	
✓ action	off	Action to apply (commonly off; may also support on)	
Key	Value	Description	

Hình 6.5 Request Params API api/device/control

Body Headers Status Code 200 OK

{ } JSON Preview Schema

success	true
message	Device command accepted
data {4}	
led	light_bulb
action	off
status	Success
timestamp	2026-03-15T00:00:00Z

Hình 6.6 Response API api/device/control